



Programa Certified AI-LLM Solution Architect

Curso Diseño de Infraestructura Escalable

Claves para construir plataformas eficientes y adaptables en la nube

www.bsginstitute.com

Andrés Felipe Rojas Parra

- Ingeniero electrónico con mas de 25 años de experiencia
- PGP Artificial Intelligence & Machine Learning at Texas University – Austin, TX
- Master en Inteligencia Artificial y Big Data – IEBSchool
- Estudiante de PGP en Cloud Computing at Texas University - Austin, TX
- Texas Executive Education Program – McCombs School of Business – Decision Science and AI Immersion Program
- Especialista en MLOps de la Universidad de Duke
- Estudiante de Deep Learning for Healthcare Specialization
- Director de Inteligencia Artificial e Innovacion Universidad EAN/Colombia



anrojas@universidadean.edu.co / arojaspa@outlook.com / arojaspa76



@arojaspa



<https://www.linkedin.com/in/arojaspa/>



+573005906373



Serverless + Alta Disponibilidad para LLMs

Capítulo 3: Arquitecturas Serverless

Capítulo 4: Reducción de Latencia y Alta Disponibilidad



¿Qué haremos en 3 horas?



0:00 – 1:00

Capítulo 3: Arquitecturas Serverless

- ¿Qué es serverless y qué NO es?
- Azure Functions vs GCP vs AWS Lambda
- Cold start: el enemigo de los LLMs



1:30 – 2:15

Capítulo 4A: Autoescalado Kubernetes

- HPA con métricas CPU/RAM
- Demo: Minikube + HPA en vivo
- KEDA: scale to zero



1:00 – 1:30

Práctica Cap. 3: Ejercicio: LLM Summarizer

- Mismo código en 3 nubes
- Deploy live a Azure Functions
- Test con cURL + Postman



2:15 – 3:00

Capítulo 4B: Pruebas de Carga

- Tipos de test: smoke, load, stress
- Demo: k6 + observar HPA escalar
- Análisis de resultados

Serverless: mitos vs realidad

✗ MITO

- No hay servidores en ningún lado
- Siempre es más barato que VMs
- Sin límites de tiempo o RAM
- Ideal para cualquier LLM
- Sin latencia adicional

✓ REALIDAD

- TÚ no gestionas los servidores
- Barato en baja carga, caro en alta
- Timeout 5-60 min, RAM hasta 32GB
- Ideal para LLMs via API externa
- Cold start: 1-15 seg la primera vez

Azure vs GCP vs AWS — Serverless para LLMs



Azure Functions

 RAM máx.	1.5 GB / 14 GB
 Timeout	5 min / Ilimitado
 Cold Start	1-3 seg
 LLM	Azure OpenAI

*Mejor integración
con Azure OpenAI*

Cloud Functions gen2

 RAM máx.	32 GB máx
 Timeout	60 minutos
 Cold Start	500ms-2 seg
 LLM	Vertex AI Gemini

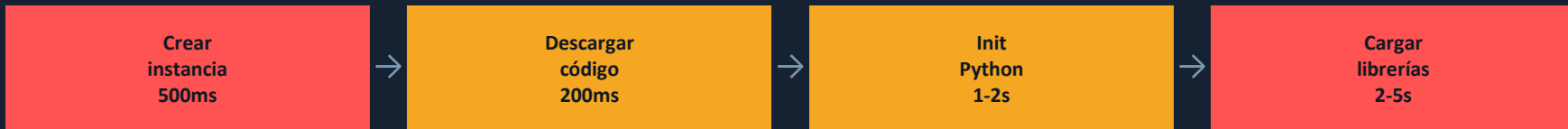
*gen2 = Cloud Run
Más flexible y rápido*

AWS Lambda

 RAM máx.	10 GB máx
 Timeout	15 minutos
 Cold Start	300ms-1 seg
 LLM	Amazon Bedrock

*Bedrock: Claude,
Llama, Titan, Mistral*

Cold Start: el enemigo de los LLMs en Serverless



Total cold start sin modelo: ~4-8 segundos | Con modelo LLM local: 30-120 segundos ❌



Min instances = 1

Siempre una instancia caliente. Costo mínimo constante.



LLM via API externa

Llamar a Azure OpenAI / Bedrock. Cold start solo del runtime (~1s).



Provisioned concurrency

AWS Lambda y Azure Premium mantienen N instancias pre-calentadas.

LLM Summarizer: mismo código, 3 nubes

```
POST /summarize {"text": "..."} → LLM → {"summary": "..."}
```

Lógica de negocio idéntica en los 3 proveedores — solo cambia el wrapper HTTP



Azure

Wrapper:

```
@app.route(route="llm-summarize")
async def llm_summarize(req):
```

Local → localhost:7071

```
func start
```

GCP

Wrapper:

```
@functions_framework.http
def llm_summarize(request):
```

Local → localhost:8080

```
functions-framework
--target llm_summarize
```

AWS

Wrapper:

```
def lambda_handler(event, context):
    body = json.loads(event['body'])
```

Local → localhost:3000

```
sam local invoke
LLMFunction
```


Reducción de Latencia y Alta Disponibilidad



● Balanceo de carga



● HPA en K8s local



● KEDA scale-to-zero



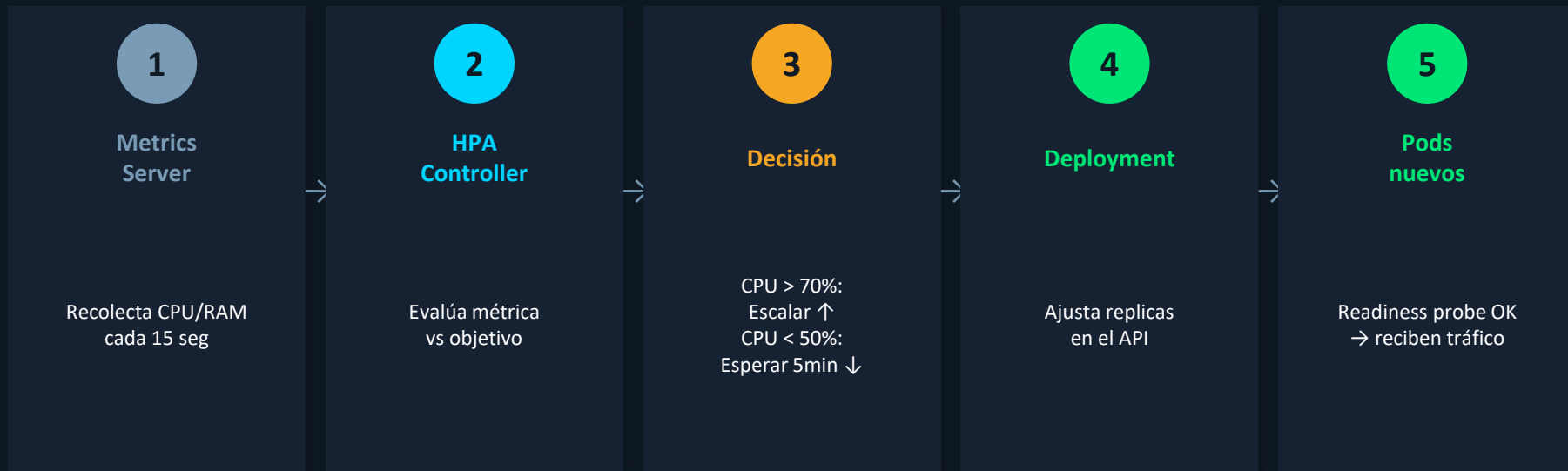
● k6 Load Testing

● Análisis de resultados

HPA: HorizontalPodAutoscaler

```
replicas_deseadas = ceil( replicas_actuales × ( métrica_actual ÷ objetivo ) )
```

Ejemplo: 2 pods × (90% CPU ÷ 70%) = ceil(2.57) = 3 pods



Demo: HPA en acción con Minikube

Comandos (en orden)

1. Iniciar Minikube

```
minikube start --cpus=4 --memory=8192
minikube addons enable metrics-server
```

2. Desplegar app + HPA

```
kubectl apply -f kubernetes/local/
```

3. Monitorear (terminal 2)

```
watch kubectl get hpa,pods -n llm-prod
```

4. Generar carga (terminal 3)

```
k6 run loadtesting/k6/load-test.js
```

Output esperado

Sin carga:

TARGETS	REPLICAS
15%/70%	2

Con carga (k6):

85%/70%	2	← escala!
85%/70%	4	← 2 pods+
65%/70%	4	← estable

Sin carga 5 min:

15%/70%	2	← baja
---------	---	--------

KEDA: Kubernetes Event-Driven Autoscaling

Característica	HPA Estándar	KEDA
Mínimo de réplicas	1 pod siempre corriendo	0 pods (scale to zero )
Métricas	Solo CPU/RAM (K8s nativo)	Cualquier métrica externa
Triggers	API de Metrics Server	HTTP, Redis, Queue, Kafka, Cron
Instalación	Incluido en K8s	Helm chart (5 min)

Ahorro de costo

Moderado

Hasta 60% en cargas variables

 Caso de uso real: API LLM usada solo en horario laboral (8h-18h)

Sin KEDA: 2 pods corriendo 24/7 = 720 pod-hours/mes

Con KEDA scale-to-zero: ~330 pod-hours/mes (noche y fines de semana = 0 pods) = 54% de ahorro

Autoescalado K8s: AKS vs GKE vs EKS



Azure AKS

HPA  Nativo

KEDA  Addon oficial

Cluster AS  Integrado

Scale-up ~5 min

★ NAP: aprovisionamiento
automático de nodos

GKE Autopilot

HPA  Nativo

KEDA Community

Cluster AS  Automático (nodos
serverless)

Scale-up <2 min

★ Pagas por pods,
no por nodos

AWS EKS

HPA  Nativo

KEDA Community

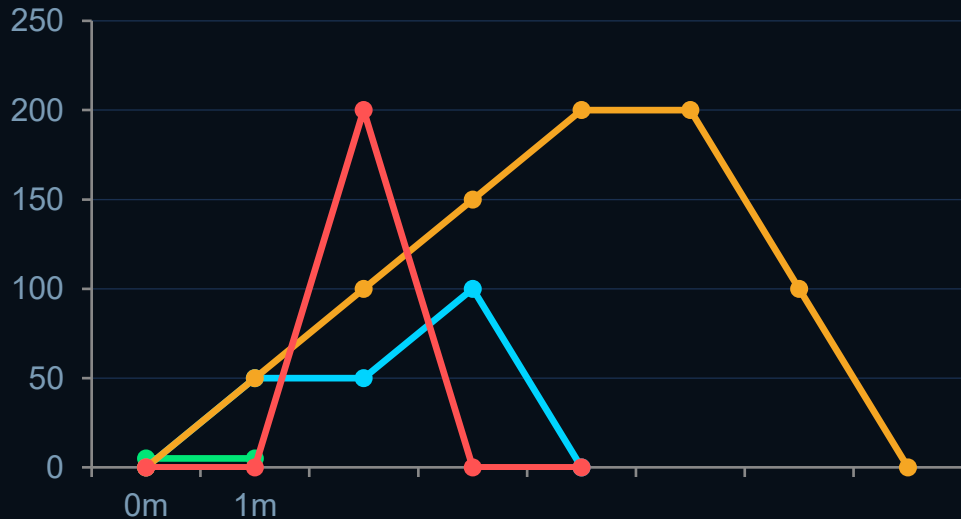
Cluster AS Karpenter (<2 min)

Scale-up <2 min
(Karpenter)

★ Spot instances: hasta
90% ahorro en nodos

Tipos de test con k6

Usuarios virtuales vs tiempo



Smoke

smoke-
test.js

5 VUs / 1 min · ¿Funciona sin errores básicos?



Load

load-
test.js

50-100 / 10 min · ¿Rinde bajo carga normal?



Stress

stress-
test.js

hasta 200 / 15m · ¿Dónde está el límite?



Spike

spike-
test.js

0→200→0 / 5 min · ¿Maneja picos súbitos?

Cómo interpretar los resultados de k6

Salida típica de k6

```
http_reqs.....: 1842  2.8/s
http_req_failed...: 1.2%  ✓ threshold <5%
http_req_duration.:
    avg=2.3s  p(50)=1.1s
    p(90)=4.2s  p(95)=6.8s
    p(99)=14.3s  max=31.2s

checks.....: 98.8% ✓ 1819 ✗ 23
cache_hits_count.: 1104 (60%)
errors.....: 1.2% ✓ < 5%

✓ RESULTADO: PASS
```

p(50) = 1.1s

La MITAD de requests responden en < 1.1s. Es la 'latencia típica' que experimenta un usuario promedio.

p(95) = 6.8s

El 95% responde en < 6.8s. El 5% más lento (cache misses, picos de CPU). Este es el SLA número.

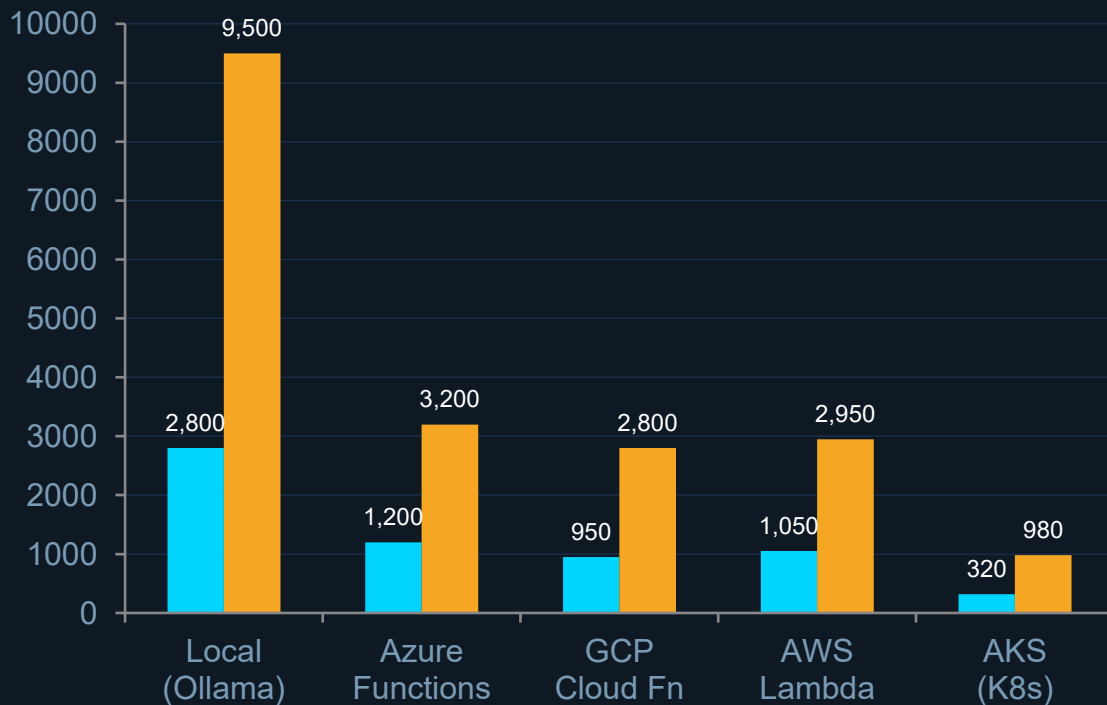
p(99) = 14.3s

El 1% más lento. Usuarios con mala suerte. Si es > 30s, revisar circuit breaker.

Cache hits = 60%

60% de requests sirvieron del cache Redis. Sin cache: todos los p95 serían > 15s.

Benchmark: Local vs Azure vs GCP vs AWS



* Valores de referencia — dependen del modelo LLM y del tráfico concurrent



AKS K8s

P95 = 980ms — el más rápido. Modelo en RAM siempre caliente, sin cold start.



Azure Fn

P95 = 3.2s — excelente para API externa (Azure OpenAI). Cold start mitigado.



Local CPU

P50 = 2.8s — lento por CPU. Con GPU: 10x más rápido.

¿Cuándo usar qué arquitectura?



Serverless puro

Usar cuando:

Tráfico impredecible, bajo costo base

Caso de uso:

API ligera con LLM externo (Azure OpenAI / Bedrock)

💰 \$0 en idle

⚡ 2-5s

📄 0 → ∞



K8s + HPA

Usar cuando:

Tráfico predecible, baja latencia crítica

Caso de uso:

LLM en RAM, respuestas < 1s, alta concurrencia

💰 \$35+/mes (nodos)

⚡ <1s

📄 2 → 20 pods

K8s + KEDA

Usar cuando:

Híbrido: ahorro + latencia

Caso de uso:

LLM con horario definido, scale to zero de noche

💰 54% vs 24/7

⚡ <1s

📄 0 → 20 pods

SESIÓN 4 COMPLETADA ✓

Lo que dominaste hoy



Serverless para LLMs: conceptos, cold start y mitigaciones



Mismo ejercicio LLM Summarizer en Azure, GCP y AWS



HPA y KEDA: autoescalado de pods en K8s local y nube



Pruebas de carga con k6: smoke, load y stress tests



Benchmark multi-cloud: cómo comparar latencia P50/P95/P99

Próxima sesión (Sesión 5):

Optimización de Costos y FinOps para LLMs — Spot instances, reservas, rightsizing, FinOps dashboard en Grafana